Persistent storage

Jarmo Hurri

Overview

- program variable memory is volatile: lost at program exit
- how to write and read program data so that it persists
- typical approaches
  - dedicated file on drive (disk)
    - serialization
    - text file
  - dedicated storage system
    - database (in great detail later)
- warnings
  - writing on your drive can destroy valuable data
  - using external libraries carries a risk

© Jarmo Hurri

Scanned with CamScanner

Computer drive as sequence of bytes

- computer has secondary storage device for persistent storage
  - solid state drive (SSD)
  - hard disk drive (HDD)
- *byte* consists of 8 bits (zeros or ones)
- abstractly drive contents is *sequence of bytes*
- these can be read or written at arbitrary locations
- following sequence would represent the word `cat`

```
s = 'cat'
print(' '.join([f'{byte:08b}' for byte in s.encode()]))
```

```
01100011 01100001 01110100
```


Serialization

- *serialization*: conversion of data into sequence of bytes
- resulting sequence can be
  - written onto drive for persistent storage
  - read from drive to restore program state (*deserialization*)
  - sent over network to other programs
- (built-in) libraries do this for you
- pros
  - easy to use: builtin support for saving any object – string, list, your own class etc.
  - very reliable
- cons
  - uses binary format: difficult to read and debug
  - object storage size unknown
  - not secure if interacting with unreliable programs

Scanned with CamScanner

Simple class used as data below ↪

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f'({self.x}, {self.y})'
```


Example 1: serialization and deserialization using `pickle` ↪

```
import pickle, pathlib # pickle for storage, hehe
file_path = pathlib.Path('data') / 'points.ser' # file on drive
points_written = [Point(x, x**2) for x in range(20)] # some data
print(points_written[-1]) # last point written

# serialize / pickle onto drive
f = open(file_path, 'wb') # open file for writing binary data
pickle.dump(points_written, f) # save data into file
f.close() # close file

# deserialize / unpickle from drive
f = open(file_path, 'rb') # open for reading binary data
points_read = pickle.load(f) # read and return data
f.close() # close file
print(points_read[-1]) # last point read
```

```
(19, 361)
(19, 361)
```

Checking if file exists ↪

- data is typically read into program if it has been saved
- natural test for this: does save file exist

```
import pathlib

# under directory data, file data.ser
file_path = pathlib.Path('data') / 'data.ser'
if file_path.exists():
    print('reading', file_path)
else:
    print('no file', file_path)
```

```
no file data/data.ser
```


Text file

- on your drive, all files are sequences of bytes
- *plain text files*, such as Python source code files
  - contain characters
  - are human-readable e.g. with text editor
  - have characters represented as bytes using specific *encoding* such as utf-8 (Unicode Transformation Format, 8-bit bytes)
  - you can check your default text encoding:

```
import locale
print(locale.getencoding())
```

- pros: easy to understand / debug / fix
- cons: object needs to be "printed" as text when written, and "parsed" back to object when read

reading it back.
-> can be a huge work if has complicated data

Scanned with CamScanner

Example 2: writing and reading data using text file ↪

```
import pathlib
file_path = pathlib.Path('data') / 'points.txt'
points_written = [Point(x, x**2) for x in range(20)] # some data
# write data onto drive: x y on each line
f = open(file_path, 'w') # open file for writing (text) data
for p in points_written: print(p.x, p.y, file=f) # print data
f.close() # close file

# read data from file
f = open(file_path) # open file (for reading text)
points_read = []
for line in f: # iterate over lines
    [x, y] = [int(word) for word in line.split()] # extract x, y
    points_read.append(Point(x, y))
f.close() # close file
print(points_written[-1], points_read[-1]) # check against orig
```

```
(19, 361) (19, 361)
```


Iterating over lines of text file

- lines of text file can be iterated over using a `for`-statement ↪

```
import pathlib
f = open(pathlib.Path('data') / 'points.txt')
for line in f:
    print(line)
f.close()
```

- for completeness, though, trailing newline character `'\n'` is preserved in lines which have it
- to get rid of trailing newline, you can e.g.

```
line = line.removesuffix('\n')
```


Scanned with CamScanner

Context manager

- Python's *context manager* is an abstraction to ensure existence of needed conditions for running block of code

  context state or environment needed by block

  manager creates and cleans up state automatically
- analogy: janitor manages context for running a school day
  - puts on lights and opens school in the morning
  - turns off lights and closes school at the end of day
- some classes provide context manager capabilities
  - most importantly: classes of objects returned by `open()`
- syntax involves keywords `with` and `as`

```
with context_manager as m:
    # block where context available
```


Example 3: closing file using context manager ↪

```
import pathlib
file_path = pathlib.Path('data') / 'points.txt'
with open(file_path) as f: # with establishes new context
    for line in f:
        print(line, end='')
# context closes, file closed automatically
```


Scanned with CamScanner

Error situations with files

- lots of things can go wrong when working with files
- reading; e.g.
  - file does not exist
  - insufficient permissions
  - wrong data format
  - file removed during reading
- writing; e.g.
  - directory does not exist
  - insufficient permissions
  - insufficient drive space
- good programs try to handle these elegantly
- *exceptions* are Python way of handling error conditions
  - program code can *raise* exception, enclosing code can *handle* it

Example 4: `try-except`

Experiment with exceptions that can be raised in following code. ↪

```
try:
    import pathlib
    file_path = pathlib.Path('data') / 'points.txt'
    with open(file_path) as f:
        points_read = []
        for line in f: # traverse lines
            # extract x, y
            [x, y] = [int(word) for word in line.split()]
            points_read.append(Point(x, y))
    print(points_read[-1])
except Exception as e: # handle any exception raised
    print('exception:', e)
```


Scanned with CamScanner

Handling exceptions

- general structure of `try-except-finally`

```
try:
    # some code
except ExceptionClass [as ...]:
    # code to handle this type of exception
# other handled exception classes
except:
    # code to handle any exception not handled above
finally:
    # code executed regardless of whether any exception was
    # raised above, or a return happened
```

- for our purposes, `with`-statement is much more useful than `finally` for cleaning up resources

Example 5: input validation

Write a program which prompts the user for a filename until the user-provided filename can be opened for writing in text mode and integer 440 has been appended to the end of file. Use

1. a boolean flag to indicate completion
2. an infinite loop with `break` when completed.

Use a context manager to close the file when done.

Scanned with CamScanner

Homework 1

1. Write a program containing a list of 10000 random integers. You can generate the random integers as you like.
2. Extend the program so that it saves the list as text and reads the list from the saved file. Use a context manager to close the file(s).
3. Add a check to your program: the list is saved only if the file does not exist. If the file exists, the list is only read.
4. Extend the program to use serialization to save the same list in another file and read the list from the file. Note: Save and read the entire list as one object.

Homework 2

Function (constructor) `float()` converts a string to a decimal number. For example

```
>>> float('1.14')
1.14
```

1. Explore which exception is raised if the string does not correspond to a decimal number.
2. Write a program which prompts the user for a decimal number until the user provides a valid input.

Scanned with CamScanner

Homework 3

Write a program which handles all special cases "elegantly" (does not crash and prints an informative message to the user) in the following:

The program prompts the user for the name of a text file, reads integers from the file and prints the average of the integers.

Special cases include: file does not exist, file can not be opened for reading, file is empty, some line on file contains text which does not represent an integer.

Homework 4

Write a function

```
filter_file(filename_in, filename_out, cond)
```

Here `filename_in` is the name of an input file, `filename_out` is the name of an output file, and `cond` is a function taking a string as a parameter and returning a boolean value. For each text line in `filename_in`, the function evaluates `cond`, and writes the line to `filename_out` exactly when `cond` returns `True`. For example, evaluating

```
def odd(s): return int(s) % 2 == 1
filter_file('data/list.txt', 'data/odds.txt', odd)
```

will write all odd integers from input file into output file.

Scanned with CamScanner

Homework continues

Handle exceptions as follows:

- any lines raising an exception when evaluating `cond` are skipped, and this is reported with `print()`
- any other exception will be printed and will result in the function returning.

Multiple files can be opened in a single `with`-statement for example as follows:

```
with (open(name1) as f1,
      open(name2) as f2):
    # block
```


Scanned with CamScanner